

Real-world (score an answer that isn't exact) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `llm_output_testing_practice.py` (Real-world section): this is the Build Task's `judge.py` in miniature — rules written first, a second model call that checks them, and a test that the judge itself gets obvious cases right. One small task (summarizing a support ticket) keeps the focus on the judge, not the pipeline. **If not:** the Build Task's judge would be the first judge you ever wrote, scoring Project 4's output with nothing to show it can be trusted.

Every version calls the real OpenAI API (`OPENAI_API_KEY` in your `.env`), so run it from inside `practice/` with `pytest llm_output_testing_practice.py -v`. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/llm_output_testing_practice.py – Real-world section
from openai import OpenAI

client = OpenAI()

TICKET = (
    "Order #4411 arrived 6 days late because of a warehouse delay. "
    "The customer wants a refund of the shipping fee."
)
SUMMARY_TASK = "Summarize this support ticket in 1-2 sentences:\n" + TICKET

def call_model(prompt):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content
```

```

def build_judge_prompt(task, answer, rules):
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_output(task, answer, rules):
    prompt = build_judge_prompt(task, answer, rules)
    reply = call_model(prompt)
    lines = reply.strip().split("\n")
    verdict = lines[0].strip().upper()
    reason = " ".join(lines[1:]).strip()
    return {"verdict": verdict, "reason": reason}

def test_summary_passes_judge():
    summary = call_model(SUMMARY_TASK)
    rules = [
        "mentions that the delivery was late",
        "mentions the shipping-fee refund request",
    ]
    result = score_output(SUMMARY_TASK, summary, rules)
    assert result["verdict"] == "PASS", result["reason"]

```

Expected output:

```
llm_output_testing_practice.py::test_summary_passes_judge PASSED
```

This works: two model calls — one writes the summary, one judges it — and the test checks the verdict, not the wording. It returns a loose dict and has no way to know whether the judge itself can be trusted — both addressed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed result, a versioned judge prompt, and a code check first

Story: "under 60 words" is a rule plain code can check for free, so it shouldn't cost a judge call. The judge handles only the rules about meaning. A dataclass result and a saved prompt version make every score readable and traceable. **If not:** you'd pay for a model call to count words, and a score from last month couldn't be tied to the judge prompt that produced it.

```
# practice/llm_output_testing_practice.py – Real-world section
from dataclasses import dataclass
from openai import OpenAI

client = OpenAI()

TICKET = (
    "Order #4411 arrived 6 days late because of a warehouse delay. "
    "The customer wants a refund of the shipping fee."
)
SUMMARY_TASK = "Summarize this support ticket in 1-2 sentences:\n" + TICKET
JUDGE_RULES = [
    "mentions that the delivery was late",
    "mentions the shipping-fee refund request",
    "does not state any fact that is not in the ticket",
]

# why: every score is tied to one exact judge prompt – change the
# prompt, change this name
JUDGE_PROMPT_VERSION = "v1"

@dataclass
class JudgeResult:
    verdict: str
    reason: str

def call_model(prompt: str) -> str:
    # how: temperature=0 – a judge should be steady, not creative
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
```

```

        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_output(task: str, answer: str, rules: list[str]) -> JudgeResult:
    prompt = build_judge_prompt(task, answer, rules)
    reply = call_model(prompt)
    # how: split once – first line is the verdict, the rest the reason
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)

def test_summary_passes_judge():
    summary = call_model(SUMMARY_TASK)
    # when: a rule code can check gets checked by code, before the judge
    assert len(summary.split()) < 60
    result = score_output(SUMMARY_TASK, summary, JUDGE_RULES)
    assert result.verdict == "PASS", result.reason

```

Expected output:

```
llm_output_testing_practice.py::test_summary_passes_judge PASSED
```

Approach 2 — a 1-5 score instead of PASS/FAIL, with a chosen bar

Story: PASS/FAIL hides how close an answer was — a "3" and a "1" both fail, but mean very different things when you're tracking quality over time. A score also needs careful reading: judges sometimes reply "Score: 4" instead of 4. **If not:** you'd lose the "almost good" signal, and a badly formatted reply would crash with a confusing `ValueError`.

```
# practice/llm_output_testing_practice.py – Real-world section
from dataclasses import dataclass
from openai import OpenAI

client = OpenAI()

TICKET = (
    "Order #4411 arrived 6 days late because of a warehouse delay. "
    "The customer wants a refund of the shipping fee."
)
SUMMARY_TASK = "Summarize this support ticket in 1-2 sentences:\n" + TICKET
JUDGE_RULES = [
    "mentions that the delivery was late",
    "mentions the shipping-fee refund request",
    "does not state any fact that is not in the ticket",
]
PASS_BAR = 4

@dataclass
class ScoredResult:
    score: int    # 1 to 5
    reason: str

class JudgeParseError(Exception):
    """The judge replied, but not in the format we asked for."""

def call_model(prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_score_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = ""
    for rule in rules:
```

```

        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer should follow:\n" + rules_text +
        "Score it from 1 (fails badly) to 5 (meets every rule).\n"
        "Reply with only the number on the first line, "
        "then one sentence explaining why."
    )

def score_output(task: str, answer: str, rules: list[str]) -> ScoredResult:
    reply = call_model(build_score_prompt(task, answer, rules))
    lines = reply.strip().split("\n", 1)
    try:
        score = int(lines[0].strip())
    except ValueError:
        # why: a named error says "the judge's format is wrong",
        # not "your answer is wrong" (Doc01's custom-error habit)
        raise JudgeParseError(
            "expected a number on the first line, got: " + lines[0]
        )
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return ScoredResult(score=score, reason=reason)

def test_summary_scores_above_bar():
    summary = call_model(SUMMARY_TASK)
    result = score_output(SUMMARY_TASK, summary, JUDGE_RULES)
    assert result.score >= PASS_BAR, result.reason

```

Expected output:

```
llm_output_testing_practice.py::test_summary_scores_above_bar PASSED
```

Approach 3 — a calibration set: answers whose right verdict you already know

Story: a judge that says PASS to everything makes every test green and protects nothing. Before trusting it, give it answers where you already know the verdict — one clearly good, two clearly bad — and check it gets all of them right. **If not:** the Build Task's "judge given an obviously bad answer scores it low" test case would have no evidence behind it, and a broken judge would wave bad answers through.

```

# practice/llm_output_testing_practice.py – Real-world section
from dataclasses import dataclass
from openai import OpenAI

client = OpenAI()

TICKET = (
    "Order #4411 arrived 6 days late because of a warehouse delay. "
    "The customer wants a refund of the shipping fee."
)
SUMMARY_TASK = "Summarize this support ticket in 1-2 sentences:\n" + TICKET
JUDGE_RULES = [
    "mentions that the delivery was late",
    "mentions the shipping-fee refund request",
    "does not state any fact that is not in the ticket",
]

# why: answers where YOU already know the right verdict – the
# judge must match you on every one before it scores anything real
CALIBRATION_SET = [
    {
        "answer": "Order #4411 arrived 6 days late due to a warehouse "
        "delay; the customer wants the shipping fee refunded.",
        "expected_verdict": "PASS",
    },
    {
        "answer": "I like pizza.",
        "expected_verdict": "FAIL",    # off-topic
    },
    {
        "answer": "Order #4411 arrived early and the customer was "
        "happy with the free upgrade to overnight shipping.",
        "expected_verdict": "FAIL",    # confident, and wrong
    },
]

@dataclass
class JudgeResult:
    verdict: str
    reason: str

def call_model(prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,

```

```

        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_output(task: str, answer: str, rules: list[str]) -> JudgeResult:
    reply = call_model(build_judge_prompt(task, answer, rules))
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)

def test_judge_calibration():
    for case in CALIBRATION_SET:
        result = score_output(SUMMARY_TASK, case["answer"], JUDGE_RULES)
        # how: the message after the comma is printed if this fails,
        # so you see the judge's own reason for the wrong verdict
        assert result.verdict == case["expected_verdict"], result.reason

```

Expected output:

```
llm_output_testing_practice.py::test_judge_calibration PASSED
```

If this fails on the pizza answer or the wrong "arrived early" summary, the judge can't be trusted yet — fix the judge prompt (clearer rules, "facts not in the ticket are a FAIL") before you let it score anything real.

Difference from Basic: Approach 1 wraps the result in a typed `JudgeResult`, saves a judge prompt version, and checks the word count with plain code instead of spending a judge call on it.

Approach 2 swaps PASS/FAIL for a 1-5 score with a pass bar, and reads the number carefully with a named `JudgeParseError`. Approach 3 doesn't change the judge at all — it tests the judge on answers whose verdict you already know, so a judge that approves everything is caught before it gates anything.

Which one should you actually write? Approach 1 is the judge to use day to day — PASS/FAIL maps straight onto a test. Switch to Approach 2's score when you want to watch quality trends, not just pass/fail. Run Approach 3's calibration test every time you change the judge prompt — it's the only evidence you have that the judge works. The Build Task's `judge.py` is Approach 1, and its `test_judge.py` is Approach 3.